本科毕业设计（论文）开题报告

毕业设计（论文）题目	基于大小模型协同的故障根因定位系统设计与实现

学　　号：	09022101
姓　　名：	马顺道
学　　院：	计算机科学与工程学院
专　　业：	计算机科学与技术
指导教师：	沈典
开题日期：	2026 年 3 月 5 日

一、　选题背景和意义：

在微服务架构已经广泛应用于互联网与企业信息系统的背景下，业务系统的服务数量快速增长，服务之间的依赖关系也变得更加复杂且动态变化。因此线上故障一旦发生，往往难以及时、准确地定位根因，排障过程对经验依赖强、成本高，这就迫切需要一种能够自动化完成故障根因定位与故障类型判定的智能分析系统来辅助运维人员。这也是本课题主要要解决的问题。

从实际问题出发，微服务故障定位的需求可以进一步分解成三个层次：第一层是对故障相关组件的"候选范围缩小"，要求在日志、指标、链路追踪等多源观测数据中快速找出可能的根因组件集合，从而避免在全量服务中盲目搜索。这对应的是多源异常信号融合下的候选召回问题。第二层是对候选组件的"证据对齐与因果推断"，要求能够结合调用链路和异常指标，识别故障传播路径，判断哪一个候选组件最符合根因特征，并同时给出合理的解释依据。这对应的是复杂证据条件下的语义推理与精判问题。第三层是对定位结果的"工程化输出"，要求系统能够稳定输出统一的结构化结果（例如根因组件、故障类型、关键证据等字段），便于后续告警联动、工单流转与复盘分析，而不是仅生成不可直接落库的自然语言长文本。

现有方法在该任务上各有局限：传统基于规则与经验的方式虽然直观，但高度依赖具体系统拓扑与业务逻辑，系统升级或业务变化会带来大量维护成本，迁移性较差；近年来智能运维 AIOps 引入机器学习与深度学习进行多源数据挖掘，在一定程度上提升了自动化能力，但仍容易受到噪声告警干扰，且解释链不足。另一方面，大语言模型在语言理解与推理方面表现突出，被尝试用于日志解释、告警摘要与故障分析，但如果直接让大模型端到端在全量组件中"推断根因"，会面临候选空间过大导致结论不稳定、推理成本较高难以支撑高频告警、以及结构化输出不稳定难以工程落地等问题。

基于上述分析，本课题在现有相关算法的基础上进行大小模型的组合创新，使其能够同时满足准确性、稳定性、推理效率与工程可落地性等方面的要求，并实现为一个可迭代、模块化的软件系统。总体思路是构建"大小模型协同"的微服务故障根因定位系统：由小模型在多源观测数据上高效完成错误信息候选召回以缩小搜索空间，再由大模型在候选集合内进行语义推理与精确判断，最终输出统一的结构化根因与故障类型结果，从而显著降低人工排障时间与经验依赖。

二、　　课题关键问题及难点：

在微服务架构的运维场景中，故障根因往往隐藏在大量噪声告警与多源观测数据之中。工程师需要同时查看平台指标、调用链、业务指标、容器信息等多类数据，才能逐步锁定根因组件并判断故障类型。若完全依赖人工排障，不仅耗时长、效率低，而且对经验依赖很强；若直接依赖大模型在全量组件范围内端到端推断，又容易出现定位结论不稳定、推理成本过高以及输出难以工程化落库等问题。因此，本课题的关键在于：如何在真实运维数据集上构建一个可解释的根因定位系统，并在定位准确度、推理效率与工程可控性之间取得平衡。

从问题本质出发，本课题的关键问题聚焦为三个方面。

第一，定位准确度不足：在微服务故障中，异常信号会沿调用链传播，导致多个非根因组件同时出现指标异常或告警；同时，多源数据之间存在采样频率、延迟与噪声差异，若缺少面向故障事件的统一组织与对齐，模型容易被“非根因告警”淹没，出现根因组件误判或同类故障结论前后不一致。比如某次故障的真实根因是上游服务错误率抬升或资源抖动，但下游多个依赖服务会同时出现延迟上升与超时告警；若直接在全部组件里推断，系统很容易把“告警最密集的下游服务”误认为根因，从而忽略上游触发点，甚至在不同窗口或不同证据组合下出现结论在上下游之间跳动的情况。因此，需要围绕故障事件将原始日志、指标、调用链等高维时间序列与结构化表格信息，转换为故障事件级别的统一表示（fault_event + evidence），并在压缩信息量的同时尽量保留区分根因与传播异常的关键证据，避免信息丢失导致后续定位失真。

第二，性能与成本压力大：运维场景下故障事件可能频繁出现，且每次分析都需要聚合多源证据；若直接将原始时间序列与日志堆叠输入模型，不仅输入冗长、推理延迟高，还会带来明显算力消耗，难以工程化部署到高频告警与告警风暴场景。比如一次故障涉及几十个组件，每个组件对应多条指标曲线与调用链片段，如果把这些原始序列全部拼接进提示词，输入会快速膨胀，导致推理延迟显著增加，甚至出现截断与冗余信息干扰；当告警风暴发生时，这种做法会让推理成本与告警频率直接相乘，时间复杂度极高，工程上不可接受。因此，需要在进入精判之前尽可能压缩候选空间与输入证据规模：通过低成本方法先筛出少量高风险候选根因组件，并对多源证据做统计摘要并将其模板化组织，让后续精判在“更小的候选集合、更短的上下文”上完成。

第三，工程落库困难：与只输出一个根因组件不同，本课题要求模型一次性输出包含多个字段的结构化结果，并能被稳定解析与落库，用于后续告警联动与复盘分析。实际中，大模型自由生成容易出现字段缺失、格式漂移、引号/括号不匹配、夹杂解释性文本等问题，从而导致“模型看似给出答案但系统无法使用”。比如同一条输入下，模型有时先输出一段自然语言解释再给 JSON，有时把字段名写成近义词（把 related_container 写成 container 或直接漏掉 kpi），有时输出多行并带代码块标记，导致解析脚本直接失败。工程上这意味着结果无法稳定写入数据库，导致无法驱动后续自动化联动。因此需要在任务定义阶段就将输出 Schema 固化为严格格式，通过提示词约束与 SFT（监督微调）让模型学习稳定输出合法 JSON；并在评估中同时考察“可解析性”和“全字段一致性”。

在上述关键问题之上，本课题的技术难点集中体现在“端到端链路中的稳定性控制”：

多源数据对齐与时间窗口选择：不同来源数据采样频率不同、延迟特性不同，窗口长度与前后偏移量如果不合理，有可能截断有效信号或引入大量噪声，从而影响候选召回与精判输入的稳定性。比如窗口过短可能只截到下游延迟上升而没截到上游错误率抬升，窗口偏移不当还可能把故障前的正常波动当成异常。

候选召回的特征工程与加权的策略：异常指标数、偏离程度、调用链错误比例、慢调用比例、业务波动等证据需要统一归一化与加权；权重调参要同时兼顾覆盖率、误报率与解释性。比如仅用“异常指标数”容易偏向“指标多的大服务”，忽略“关键但指标少的小服务”。

精判输入构建与上下文长度控制：若直接堆砌原始日志与时间序列会导致输入过长与冗余，需要将时间序列转换为统计摘要（均值、极值、分位数、错误率），并通过模板化提示词把证据组织成格式一致、易理解的指令输入。比如将“每分钟延迟曲线”摘要为“峰值/上升斜率/持续时间/是否伴随失败率上升”，比直接粘贴长序列给大模型精判更利于稳定判断。

结构化输出约束与解析稳定性：需要通过“只输出单行 JSON”的约束设计、健壮的后处理解析与异常检测机制，将解析失败显式纳入评估并形成可回归的质量门槛。比如根因判断方向正确但输出夹带解释文本导致 JSON 解析失败，工程上仍然不可用。

微调稳定性和资源约束下的数值问题：在 4-bit QLoRA 方案下，量化配置、学习率、梯度裁剪、精度设置等组合稍有不当就可能引发训练不稳定，需要结合训练日志与评估结果定位问题来源并形成可复现的稳定配置。比如学习率过大可能导致损失震荡并带来输出格式漂移，量化或精度设置不当可能导致训练中断或评估结果大幅波动。

结果解释：当不同指标表现不一致时，需要评估脚本输出样本级对比（预测 vs 真实标签），结合典型案例分析误差模式，然后反向指导候选召回权重到小模型、提示词模板到大模型与数据构造策略的迭代。

注：开题报告可单独装订，但在院（系）范围内，封面和装订格式必须统一。

三、文献综述（或调研报告）：

3.1 传统根因定位方法研究现状

早期的根因定位工作多依赖规则与专家系统，通过人工编写告警抑制规则、故障模式库以及拓扑推断规则来输出根因结论。这类方法实现成本相对较低，且由于规则是人为显式定义的，因此解释性较强，便于运维人员理解与复盘。但其局限也很明显：一方面，规则需要持续维护，系统一旦升级、服务拆分或告警策略变化，原有规则就可能失效；另一方面，规则往往依赖特定业务与拓扑结构，迁移到其他系统时需要大量重写，难以满足跨系统复用的需求。

在规则方法之外，也有研究利用图模型与概率推理方法进行根因定位，例如基于服务依赖图构建贝叶斯网络或马尔可夫随机场，在图上建模故障传播关系并推断最可能的根因节点。这类方法在拓扑相对稳定、数据质量较高的场景下具有一定优势，能够将“传播关系”显式编码进模型结构中，从而在理论上更贴合微服务故障传播的机制。但该方向通常对模型构建、结构设计与参数学习提出较高要求，且当拓扑频繁变化或观测噪声较高时，推断结果会受到显著影响，工程落地难度较大。

3.2 机器学习与深度学习方法研究现状

随着智能运维研究的推进，国内外大量工作尝试将机器学习与深度学习方法引入根因定位任务，典型方向主要包括三类。

第一类是基于时间序列的异常检测与根因推断方法。这类方法通常对监控指标进行建模，通过聚类、统计异常检测或基于 LSTM、CNN 等深度模型识别异常时段与异常指标，再将异常信号映射到可能的组件集合，从而得到“最可疑的组件”排序。该方向的优势在于能够从历史数据中学习异常模式，并在固定系统与稳定数据分布下取得较好的局部准确率。

第二类是基于拓扑的图神经网络方法。该类工作往往将服务或容器视为图节点、调用关系视为边，在服务依赖图或调用图上进行信息传播与聚合，以实现故障定位与传播路径分析。该类方法能在一定程度上利用结构信息提升定位效果，但通常隐含拓扑相对稳定的假设，并且网络结构与超参数往往是在特定拓扑下调优得到，当系统重构或依赖关系变化明显时，需要重新建模与调参。

第三类是多任务或联合学习方法。该方向试图构建统一模型，同时完成告警关联、故障检测、故障类型分类与根因定位等任务，以期通过共享表示提升整体效果。但在真实工程中，多任务训练往往对数据标注规模与一致性提出更高要求，且当任务目标之间存在冲突时，模型训练与评估会变得更复杂。

综合已有研究可以发现，机器学习与深度学习方法虽然提升了自动化程度与一定场景下的准确率，但在工程实践和跨系统应用中普遍面临共性短板：其一，模型定制成本高，对特定指标体系、采样频率与特征工程强依赖；其二，解释性有限，输出往往是异常分数或概率，难以直接回答为什么是这个组件；其三，跨系统迁移能力不足，往往需要重新做一次特征工程、建模与调参，很难做到少量样本的快速适配。这些问题在微服务拓扑动态变化、告警噪声高的环境下尤为突出。

3.3 大语言模型在运维场景中的应用研究现状

近年来，GPT、Qwen 等大语言模型开始被引入运维场景，典型应用包括日志解析与摘要、告警合并与归类、自然语言解释生成以及对话式排障助手等。该类研究展示了大语言模型在语义理解、信息组织与解释生成方面的潜力，尤其适合将分散的告警与证据组织成可读的分析结论，从而降低一线人员的理解成本。

但从精确、可控地输出结构化根因信息的角度看，现阶段单纯依赖大模型进行端到端根因定位仍存在明显挑战：当候选空间过大时，模型容易给出不稳定甚至错误的结论；模

型输出倾向于生成自由文本，难以严格遵守字段约束；在要求输出固定格式（如单行 JSON）时，常出现字段缺失、格式漂移、引号不匹配等问题，导致结果不可解析、不可直接接入工程系统。此外，大模型推理成本相对较高，在高频告警场景下直接使用会带来资源消耗压力。因此，大模型在运维中的应用需要与工程约束相结合，不能仅停留在能解释层面，还需要解决能稳定产出可用结构化结果的问题。

3.4 大小模型协同的研究趋势

在推荐系统与搜索排序等领域，召回与精排的两阶段架构已经被长期验证：轻量模型负责在大规模候选空间中进行高效召回，复杂模型负责在候选集合内做精细判断，从而在效果与成本之间取得平衡。将这一思想迁移到根因定位任务中，可以将问题拆分为候选根因召回和候选内精判两步：前者强调覆盖率与效率，后者强调准确率与解释能力。

与传统单模型端到端相比，大小模型协同的核心价值在于可以显著缩小大模型的推理范围，降低幻觉风险与推理成本，并让评估更可控：小模型负责输出 Top-K 候选集合，能够用 Hit@K 等指标量化召回能力；大模型只在候选集合内推断根因与故障类型，能够在更严格的输入约束下提升输出稳定性。该方向虽然近年来逐渐兴起，但仍缺乏面向真实运维数据、可复现的系统化流程与评价体系，这也构成本课题的主要研究切入点。

3.5 工具增强与智能体协同相关研究进展

除大小模型协同外，近期研究也开始关注通过工具增强与智能体协同提升复杂任务中的推理稳定性与可验证性。

一类工作强调将推理过程显式建模为可调用工具的智能体行为。例如 AgentThink 框架将推理链与动态工具调用结合，使模型能够在推理过程中判断是否需要借助外部工具获取额外信息，并通过结构化样本构造包含推理、工具、验证信息的训练数据；其训练策略采用监督微调与强化学习相结合的流程，用多维度约束提升推理一致性与决策准确率。该类研究的启示在于：复杂任务的可靠性提升不仅来自更强的语言生成能力，也来自可交互、可验证的信息获取与中间校验机制。

另一类工作强调多智能体协作的系统设计。例如 MABC 将根因分析拆分为告警解析、依赖分析、候选生成与证据校验等多个智能体，并借鉴区块链共识机制，通过投票与确认降低单一视角偏差导致的错误输出；FlowXpert 则从流程编排角度出发，强调通过知识库约束与协同演化优化排障路径，以减少冗余步骤并提升长期运行稳定性。

此外，也有研究直接聚焦大模型与小模型分类器协同的根因分析模式。例如 One-Shot Failure Root Cause Analysis 提出在标注稀缺条件下由大模型负责语义理解与候选生成、小模型负责快速判别筛选，从而在极低样本下提升稳定性；RCAgent 面向真实云环境，将大语言模型作为控制智能体，通过工具调用分析日志、代码与历史知识，并引入上下文压缩机制与 JSON 修复和错误处理策略，提升在工业级场景下的可用性与鲁棒性。

这些工作从不同角度表明，在噪声高、数据规模大且约束复杂的运维场景中，仅依赖单次生成式推理往往不够可靠，需要引入候选约束、工具校验、多智能体分工、结构化输出修复等系统设计来提升整体可用性。对本课题而言，这些研究为后续在大模型精判阶段进一步引入约束、校验与流程化设计提供了明确参考。

3.6 面向本课题的综合分析与设计取舍

结合上述国内外研究现状可以看出，传统规则与图模型方法在解释性或传播建模方面具有一定优势，但维护成本高、迁移性差；机器学习与深度学习方法提升了自动化与局部准确率，但对特征工程与特定场景强依赖，且解释性与跨系统迁移不足；大语言模型具备更强的语义理解与解释能力，但在候选空间大、输出需结构化的任务中面临稳定性与工程可控性挑战。

因此，本课题选择采用统计小模型与大语言模型的协同框架：统计学小模型在多源指

标与调用链证据上进行加权评分与排序，生成根因 Top-K 候选集合，重点保证召回能力与可评估性；大模型在候选集合内结合证据摘要进行语义推理与精判，重点提高精度与解释能力，并通过监督微调与模板约束提升结构化输出稳定性。

参考文献：

[1] Qian K, Jiang S, Zhong Y, et al. AgentThink: A Unified Framework for Tool-Augmented Chain-of-Thought Reasoning in Vision-Language Models for Autonomous Driving[C]//Findings of the Association for Computational Linguistics: EMNLP 2025. Suzhou, China: Association for Computational Linguistics, 2025: 10663-10682.

[2] Zhang W, Guo H, Yang J, et al. mABC: multi-Agent Blockchain-Inspired Collaboration for root cause analysis in micro-services architecture[C]//Findings of the Association for Computational Linguistics: EMNLP 2024. 2024: 4017-4033.

[3] Han Y, Du Q, Huang Y, et al. The potential of one-shot failure root cause analysis: Collaboration of the large language model and small classifier[C]//Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. 2024: 931-943.

[4] Shi B, Luo Y, Wang J, et al. FlowXpert: Expertizing Troubleshooting Workflow Orchestration with Knowledge Base and Multi-Agent Coevolution[C]//Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2. 2025: 4839-4850.

[5] Wang Z, Liu Z, Zhang Y, et al. Rcagent: Cloud root cause analysis by autonomous agents with tool-augmented large language models[C]//Proceedings of the 33rd ACM International Conference on Information and Knowledge Management. 2024: 4966-4974.

[6] Soldani J, Brogi A. Anomaly Detection and Failure Root Cause Analysis in (Micro)Service-Based Cloud Applications: A Survey[EB/OL]. arXiv:2105.12378, 2021-05-26.

[7] Du M, Li F, Zheng G, et al. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning[C]//Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS 2017). Dallas, TX, USA: Association for Computing Machinery, 2017: 1285-1298.

[8] He P, Zhu J, Zheng Z, et al. Drain: An Online Log Parsing Approach with Fixed Depth Tree[C]//Proceedings of the 2017 IEEE International Conference on Web Services (ICWS 2017). Honolulu, HI, USA: IEEE, 2017: 33-40.

[9] Zhu J, He S, Liu J, et al. Tools and Benchmarks for Automated Log Parsing[C]//Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP 2019). Montréal, QC, Canada: IEEE, 2019: 121-130.

[10] Zhang X, Xu Y, Lin Q, et al. Robust Log-Based Anomaly Detection on Unstable Log Data[C]//Proceedings of the 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2019). Tallinn, Estonia: Association for Computing Machinery, 2019: 807-817.

[11] Pham L, Zhang H, Ha H, et al. RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems with Telemetry Data[EB/OL]. arXiv:2412.17015, 2024-12-22.

[12] Wu L, Tordsson J, Elmroth E, et al. MicroRCA: Root Cause Localization of Performance Issues in Microservices[C]//Proceedings of the 2020 IEEE/IFIP Network Operations and Management Symposium (NOMS 2020). Budapest, Hungary: IEEE, 2020: 1-9.

[13] OpenTelemetry. OpenTelemetry Specifications[EB/OL]. 2025-10-17[2026-02-26].

四、　方案（设计方案、或研究方案、研制方案）论证

本课题面向微服务系统中的故障根因定位与故障类型判定任务，拟在多源异构运维数据（平台指标、调用链、业务指标、容器信息等）的基础上，构建“统计小模型 + 大语言模型”的协同方案。整体实现采用 Python 开发，模型训练与推理基于主流大模型框架完成。

4.1 总体设计思路与目标

本课题针对第 2 节提出的三个核心问题，即**定位准确度不足、推理效率与成本压力大、工程落地困难**，将一次故障定位过程组织为"故障事件级"的分析任务。总体思路是：围绕故障事件构造统一分析窗口，对多源观测数据进行裁剪、对齐与摘要，先利用统计小模型在全量组件上完成候选召回，再将大模型推理约束在受控候选空间内执行精细判定，最终输出可直接解析和使用的结构化结果。这样既能够减少无关噪声和冗余输入，又能够压缩大模型搜索空间，并保证输出满足工程系统的结构化要求。

从问题对应关系来看，本课题提出的三项关键设计并不是彼此孤立的技术堆叠，而是分别针对前述三个核心问题形成有针对性的解决路径。

其一，围绕故障时间点构造事件级分析窗口，对平台指标、调用链、业务指标和容器信息等多源数据进行裁剪、对齐与摘要，并生成组件级证据表示。这一设计之所以能够对应解决“定位准确度不足”的问题，是因为微服务场景中的故障传播通常会引发大量次生异常和伴随告警，若直接输入原始全量数据，模型容易同时看到真实根因信号与大批传播性噪声，从而把受影响组件误判为根因组件。通过事件级窗口约束，可以把分析范围收缩到故障发生前后的关键时段；通过多源数据对齐，可以把原本分散、异步的观测信息统一到同一故障上下文中；通过证据摘要，可以进一步压缩冗余序列、突出异常强度、变化方向和关联组件等关键信息。换言之，该设计的作用在于先提升输入证据的相关性与一致性，再交由后续模块判断，从而降低噪声干扰，提高根因识别的准确性。

其二，基于可解释统计特征对全量组件进行打分排序，输出前 K个根因候选集合，以较低计算代价尽可能覆盖真实根因，并给出可解释的排序依据。这一设计之所以能够对应解决“推理效率与成本压力大”的问题，是因为若直接将全量组件及其相关观测输入大模型进行端到端推断，大模型不仅需要处理更长的上下文，还需要在更大的搜索空间内完成判断，因而会显著增加推理延迟与算力开销。通过统计小模型先完成候选召回，可以在代价较低的条件下对组件进行预筛选，把后续大模型的判断范围从"全量组件空间"压缩到"少量高相关候选空间"。这样，大模型不再承担大范围粗搜索任务，而只需在更小的候选集合内执行精细判定，既降低了推理成本，也减少了无约束搜索带来的不稳定性。因此，这一设计的关键作用在于用小模型完成高召回的前置筛选，用候选压缩换取大模型推理效率的提升。

其三，在候选集合内引导大模型结合证据进行推理，并稳定输出符合统一约束的单行 JSON，其中固定包含 root_cause_component、fault_type、kpi、related_container 四个字段，从而保证输出可解析、可落库、可联动。这一设计之所以能够对应解决“模型结果难以解析、难以落地使用”的工程问题，是因为工程系统真正需要的并不是开放式自然语言描述，而是能够被程序直接接收、校验、存储和调用的结构化结果。如果缺少统一输出约束，大模型容易产生解释性冗语、字段缺失、键名漂移或格式错误，从而导致结果虽然"看起来合理"，却无法进入后续告警联动、结果落库与错误复盘流程。通过在候选集合内进行受控

推理，并对输出模式施加统一 Schema 约束，可以显著提升结果格式的一致性和可解析性；同时，通过固定字段定义，系统也能够对结果进行自动校验与量化评估。因此，这一设计的核心作用并不仅在于"让模型输出答案"，而在于"让模型输出能够直接进入工程流程的标准化答案"。

在上述总体思路下，本课题的完整方法链路可进一步分解为四个环节。

第一， 围绕故障事件组织多源数据，形成统一的故障级表示。

系统以故障事件为中心构造分析窗口，对平台指标、调用链、业务指标及容器信息等多源数据进行裁剪与对齐，并将关键证据以统计摘要方式表示，形成统一的故障级输入表示，即 fault_event + evidence。这一过程的核心目的在于减少原始序列直接堆叠所带来的上下文冗余和噪声干扰，使后续模块能够基于相对稳定、紧凑的事件级证据进行分析。该步骤主要面向定位准确度问题，通过增强证据表达的一致性，减弱传播异常和次生异常对根因识别的干扰。

第二， 小模型执行可解释的 Top-K 候选召回。

在统一证据表示的基础上，系统利用多种可解释统计特征对全量组件进行评分排序，输出少量根因候选集合。这里的小模型并不直接给出最终定位结论，而是承担"高召回、低成本"的前置筛选角色，其目标是在计算代价较低的条件下尽可能保证真实根因进入候选集合，并保留候选排序的可解释性与可复核性。组件评分可表示为：

$$Score(s) = \sum_{i=1}^{m} w_i \cdot z_i(s)$$

其中，$z_i(s)$ 表示组件 s的第 i个归一化统计特征，w_i 表示对应权重参数。按 $Score(s)$ 从高到低排序即可得到 Top-K 候选集合。该步骤主要对应解决推理效率与成本问题，通过前置候选压缩，将后续大模型推理约束在较小且更可能包含真实根因的搜索空间内。

第三， 大模型在候选集合内完成精细判定与多字段联合预测。

在小模型给出的 Top-K 候选基础上，系统选用 Qwen2.5-7B-Instruct 作为基座模型，在候选集合内结合故障事件与证据摘要执行候选内精细推理，并一次性输出根因组件、故障类型、关键指标与相关容器四个字段。为避免模型重新回到全量组件空间中进行无约束推断，系统通过模板化提示形式明确限定其判断范围，只允许其在候选集合内完成选择和字段生成。该步骤既利用了大模型在跨模态证据整合和多字段联合预测上的优势，又通过候选约束保证了推理过程的可控性。它既进一步提升定位准确度，也与前一步共同缓解了推理成本过高的问题。

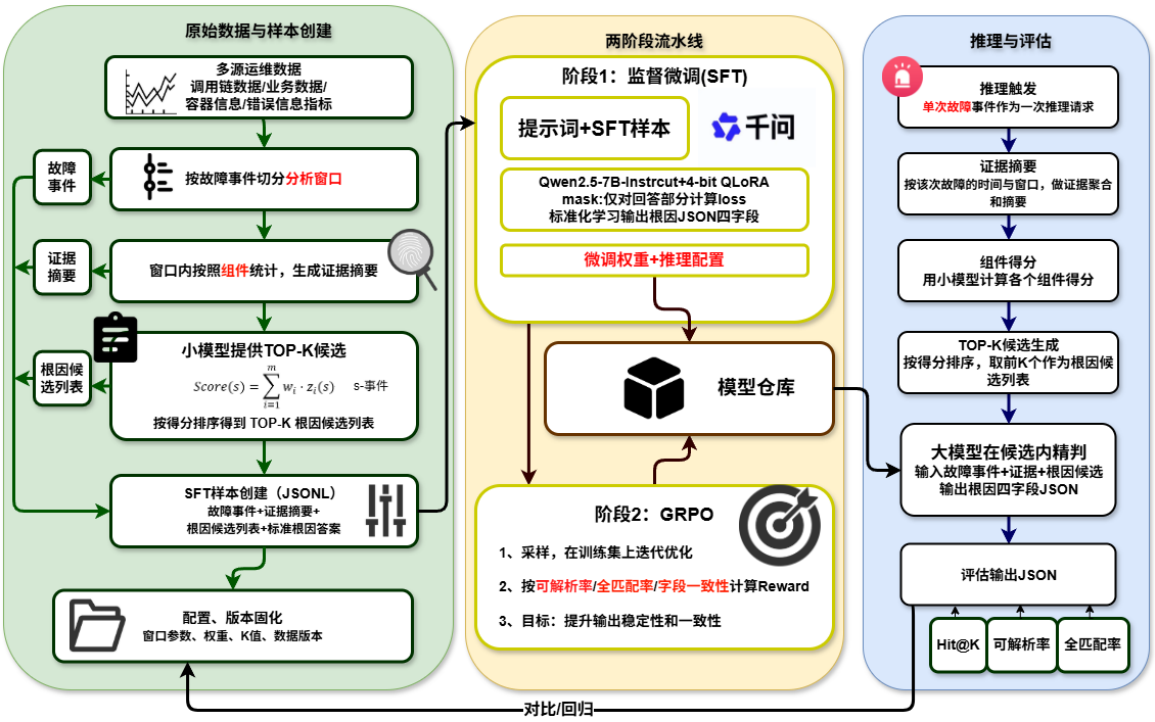
第四，通过统一结构化输出与评估体系完成结果验证与迭代回归。

系统对大模型输出施加统一的结构化约束，并通过解析校验机制确保输出满足单行 JSON 的格式要求。在评估阶段，系统从三个层面衡量方法效果：一是根因定位效果，包括候选覆盖能力与候选内精判效果；二是故障类型、关键指标及相关容器等字段的预测质量；三是工程可用性，例如可解析率、全字段一致性等。在此基础上，系统进一步对候选规模 K、证据摘要策略、提示模板以及训练配置进行可回归迭代优化。该步骤主要对应解决工程落地问题，保证模型输出不仅"有结果"，而且"可解析、可落库、可比较、可复现"。

与仅依赖传统规则方法或单一模型直接推断的方案相比，本课题采用“统计小模型候选召回 + 大模型候选内精判”的协同设计，将大模型能力约束在可控候选空间内，通过证据摘要降低输入噪声，通过候选召回压缩搜索范围，通过结构化输出提高工程可用性，从而在定位准确度、推理效率与工程落地性之间取得较为平衡的效果。同时，该方案也为后续进一步引入更复杂的优化训练机制或外部工具增强策略预留了扩展空间。

4.2 系统架构与模块设计

示意图：系统架构层次与模块



结合 4.1 节提出的总体设计思路及工程实现需求，本课题围绕故障根因定位任务的完整处理过程，对系统进行分层设计，整体构建为**原始数据与样本创建、两阶段训练流水线、推理与评估**三个相互衔接的功能区域。三部分通过统一的数据对象、参数配置及模型版本进行关联，形成“样本构建–模型训练–在线推理–结果评估–对比回归”的闭环体系。该架构既体现了统计小模型与大模型在功能上的分工关系，也保证了训练阶段与推理阶段在数据组织方式和候选约束机制上的一致性。

（1）原始数据与样本创建模块

原始数据与样本创建模块对应图中左侧部分，主要任务是将多源运维原始数据转换为可供模型训练使用的结构化样本。该部分首先接收平台指标、调用链数据、业务数据、容器信息、错误日志等多源记录，并以故障事件为中心组织后续处理流程。系统根据故障时间点构造分析窗口，确定窗口起止范围及该窗口内参与分析的数据片段，从而将连续的原始时序数据转化为面向单个故障事件的分析单元。

在分析窗口确定之后，系统对窗口内各组件相关的指标、调用链与业务信息进行统计处理，生成故障事件对应的证据摘要。在此基础上，统计小模型在“特定故障事件–特定时间窗口”的条件下，读取组件级统计特征及权重配置，对候选组件逐一计算得分，并按照得分高低生成前 K 个根因候选列表。该过程的目标并非直接给出最终根因，而是以较低计算开销完成候选空间收缩，为后续大模型精细判断提供受控范围。

随后，系统进一步构建监督微调样本。样本采用 JSONL 形式组织，主要包含故障事件、证据摘要、小模型生成的候选列表以及标准答案等内容。换言之，该模块并非仅对标签进行简单整理，而是将故障事件、证据上下文与候选空间共同组织成统一输入，使模型在训练阶段能够学习"在候选约束下依据证据完成结构化预测"的任务模式。最后，系统对窗口参数、特征权重、候选规模 K值及数据版本等配置进行固化，以支撑后续实验复现、结果回归与不同版本之间的对比分析。

（2）两阶段训练流水线模块

两阶段训练流水线模块对应图中中间部分，是本课题中大模型能力构建与持续优化的核心环节。该部分由监督微调阶段和强化优化阶段组成，分别承担基础能力学习与输出质量提升两类功能。

在第一阶段中，系统将左侧模块构建得到的结构化训练样本输入基座模型进行监督微调。由于训练样本中已经包含故障事件、证据摘要以及前 K个候选组件，因此模型学习的目标并非在全量组件空间中自由生成结果，而是在候选约束下输出包含根因组件、故障类型、关键指标与相关容器的结构化结果。具体实现上，系统选用 Qwen2.5-7B-Instruct 作为基座模型，并采用 4-bit QLoRA 方式进行参数高效微调。训练过程中仅对回答部分计算损失，对提示部分进行掩码处理，从而突出模型对结构化字段预测能力的学习。该阶段的输出主要包括微调后的模型权重及对应推理配置。

在第二阶段中，系统在监督微调模型的基础上进一步引入 GRPO 优化机制，用于提升模型输出的稳定性、一致性及工程可用性。其基本过程为：针对同一输入采样生成多条候选输出，并围绕可解析率、全字段匹配情况以及字段一致性等指标构造奖励信号，再依据奖励结果对模型策略进行更新。与监督微调阶段相比，GRPO 更强调对输出质量的进一步约束，尤其关注结构化结果的格式稳定性、字段完整性以及候选约束条件下的预测一致性。经该阶段优化后的模型同样写入模型仓库，与监督微调阶段产物共同构成可比较、可回滚、可管理的模型版本集合。

模型仓库在整个系统中承担统一存储与版本管理的作用。其一方面保存不同阶段、不同实验配置下的模型权重与推理参数，另一方面为在线推理阶段提供按版本加载能力，从而保证训练产物能够被稳定地部署到推理端使用，并支持后续的版本追踪与效果对比。

（3）推理与评估模块

推理与评估模块对应图中右侧部分，用于将训练得到的模型应用于单次故障事件定位任务，并完成结果校验与性能评估。该模块以单次故障事件作为推理触发条件。系统首先围绕该故障事件构造分析窗口，并对窗口内相关多源数据进行统计与聚合，生成对应的证据摘要。随后，系统继续采用与训练阶段一致的统计特征和权重配置，对各组件进行打分，并生成当前故障事件对应的前 K个根因候选列表。由此可见，推理阶段在证据形成方式和候选生成策略上与训练阶段保持一致，这有助于减小训练输入分布与在线推理输入分布之间的偏差。

在候选列表生成之后，系统从模型仓库中加载指定版本的大模型，将故障事件、证据摘要及候选列表共同输入模型，执行候选空间内的精细判定，输出结构化结果。与直接面向全量组件的开放式生成相比，该方式能够更集中地利用大模型的综合推理能力，并在一定程度上降低无约束输出带来的不稳定性。

模型输出之后，系统还需执行结构化校验与评估。首先，对输出结果进行格式检查，确认其是否能够被正确解析，并验证四个目标字段是否完整。随后，系统依据真实标签计算相应评估指标，例如候选覆盖能力、结果可解析率以及全字段匹配率等。评估结果与结构化输出一并保存，并通过图中底部的对比回归链路反馈至参数与版本管理模块，为后续权重调整、候选规模调整、模型版本选择及实验结果解释提供依据。因此，推理与评估模块不仅承

担在线定位任务，也承担系统持续迭代优化的反馈职能。

（4）模块间关键衔接关系

从整体架构看，各模块之间的关键衔接主要体现在以下三个方面。

其一，**样本构建与模型训练之间的衔接**。左侧模块最终形成的训练样本并非简单的标签集合，而是已经融合了故障事件、证据摘要、小模型候选列表与标准答案的结构化输入输出样本。该样本直接送入中间训练模块，使大模型在训练阶段即学习"基于证据并受候选约束完成结构化预测"的任务范式。

其二，**训练阶段与推理阶段之间的衔接**。中间模块产出的模型权重与推理配置统一写入模型仓库，推理模块通过仓库按版本加载模型执行候选内精判，从而实现训练与部署之间的规范化连接，并支持后续的版本比较与回滚。

其三，**推理结果与系统回归之间的衔接**。右侧模块在生成结构化输出的同时，对候选覆盖效果与结构化可用性进行统计，并将结果反馈到配置固化与实验记录中，使得不同候选规模、权重设置、模型版本和训练策略之间能够进行系统性的对比分析。

（5）关键输出对象与约束

为保证模块之间衔接清晰、接口稳定，系统在实现层面对若干关键数据对象进行了统一约定。故障事件对象用于描述单次故障的基本信息，包括故障时间、分析窗口及对象标识等；证据摘要对象用于表示窗口内按组件聚合后的统计结果；候选列表对象用于表示按得分排序得到的前 K 个根因候选；最终输出对象为结构化 JSON，其中固定包含 root_cause_component、fault_type、kpi、related_container 四个字段。

其中，最终输出的结构化 JSON 必须通过校验后方可进入存储与评估流程。对于无法解析、字段缺失或格式漂移的输出，系统将予以记录并纳入可解析率统计。这样可以保证系统不仅关注预测准确性，也关注工程环境下的结果可落库性与可用性。

（6）评估与回归记录

为了使系统架构具备可验证性与可迭代性，本课题在当前阶段重点规划两类核心指标。第一类指标为候选覆盖能力，用于衡量候选召回模块是否能够将真实根因包含在前 K 个候选集合中，其定义为

$$Hit@K = \frac{1}{N}\sum_{j=1}^{N} \mathbf{1}(y_j \in Cand_j^{(K)})$$

其中，$Cand_j^{(K)}$ 表示第 j 个故障事件对应的前 K 个候选集合，y_j 表示该事件的真实根因组件。

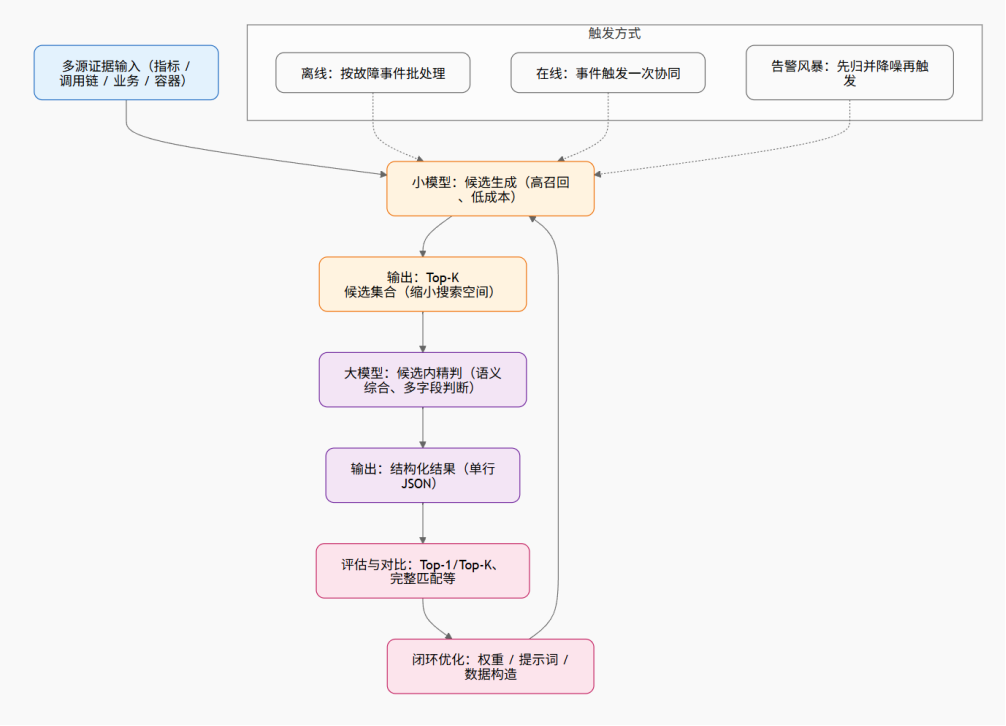
第二类指标为结构化可用性，用于衡量模型输出是否能够被系统正确解析并进入后续流程，其定义为

$$ParseRate = \frac{\#\text{可解析为合法 JSON 的输出}}{\#\text{总输出}}$$

在此基础上，系统还可进一步统计全字段匹配率等指标，用于评价模型在结构化预测任务上的整体表现。上述指标连同模型版本、配置哈希、数据版本及实验参数一起归档保存，以支撑不同候选规模 K、不同召回配置及不同训练策略之间的对比分析与结果回归。

4.3 大小模型协同

示意图：大小模型协同关系



（1）协同分工的合理性

· 小模型基于统计特征与可解释的加权规则，作为高召回、低成本的候选生成器，用于快速缩小搜索空间。

· 大模型具备更强的语义理解与推理能力，计算成本高，作为对有限候选集合的精判器。

· 将二者组合后，搜索空间由全量组件压缩到 K 个候选，降低大模型负担；同时利用大模型对候选进行语义综合与多字段判断，提高整体精度与输出质量。

（2）协同频率与触发时机设计

· 离线实验阶段以**历史故障事件**为单位批处理。

· 在线部署采用**事件驱动**方式：当产生新的故障告警时触发一次协同流程，并输出结构化定位结果。

· 面对告警风暴场景，将在协同前增加告警归并与降噪策略，减少重复计算。

4.5 故障事件与输出结果示例

以下给出**一条真实样本**的输入要点与标准输出，便于与后文"故障判断流程"对照理解。

故障事件（fault_event）示例（对应某次 CPU 类故障注入）：

- index：113
- object：docker
- fault_description：CPU fault
- kpi：container_cpu_used
- name：docker_006（即真实根因组件）
- container：container_002
- log_time：2020-05-23 04:47:00

- duration：故障持续时间（由数据集给出）
- window_start / window_end：本课题以故障时间为中点构造的分析窗口起止时间

证据（evidence）：时间窗口内平台指标（container_cpu_used）、调用链指标、业务指标的**统计摘要**（条数、均值、最大/最小值、失败率、慢调用率），按组件/服务聚合后作为"证据摘要"输入模型。

小模型给出的 Top-K 候选（top_candidates）
（示例）：["docker_004", "docker_002", "docker_001", "docker_003", "docker_006"]。大模型必须从该列表中选出根因组件。

标准输出（output / 标签）：

```
{
  "root_cause_component": "docker_006",
  "fault_type": "CPU fault",
  "kpi": "container_cpu_used",
  "related_container": "container_002"
}
```

4.6 故障判断流程

步骤	输入	输出/结果
故障触发	监控/注入记录	fault_event（index=113，CPU fault，kpi=container_cpu_used，name=docker_006，container=container_002，log_time=2020-05-23 04:47:00）
窗口与对齐	fault_event（log_time）	window_start/end（±15min）；筛出窗口内各组件的指标/链路/业务原始记录
证据摘要	窗口内原始记录	evidence（按组件统计聚合：指标统计、失败率/慢调用率、业务汇总等）
小模型召回	fault_event + evidence	top_candidates（示例：[docker_004,…,docker_006]，包含真实根因）
构造输入训练	fault_event + evidence（证据摘要）+ top_candidates（小模型召回的）	评估集：instruction（提示词）+input（模型用来推理的内容）+output（标准答案）； 训练集：instruction（提示词）+input（推理内容）
大模型精判	instruction + input	单行 JSON 预测（四字段：根因/类型/KPI/容器）
解析评估	预测 JSON + 标签 output	解析与逐字段比对：Parse Rate、Top-1/3/5、各字段 Acc、Full Match
后端 API	预测 JSON	供前端使用的标准接口

4.7 后续工作

后续工作主要围绕"数据–模型–系统–评估"四条主线展开，在已有研究目标与三项研究内容的基础上进一步细化为可执行的推进计划。

第一，完善数据准备与样本构建流程。后续将继续补齐多源观测数据的对齐与清洗逻辑，统一故障事件级的时间窗口构造方式，完善证据摘要字段与统计口径，确保不同日期、不同故障类型样本在输入结构上保持一致。同时对现有样本进行质量抽检与分布统计，处理标签噪声、类别不均衡与字段缺失等问题，并迭代形成更稳定的监督微调数据集，以支撑后续训练与对比实验。

第二，完成大模型监督微调与结构化输出稳定化。后续将在 QLoRA 等参数高效微调方案下继续进行 SFT 训练，重点优化提示词模板与输出约束，使模型能够稳定生成字段齐全、可解析的结构化结果。训练过程中将对关键超参数（学习率、batch size、梯度累积、量化配置等）进行系统化对比，形成可复现的训练配置与日志记录，并通过典型案例分析定位模型错误模式，为数据构造与提示词设计提供反馈。

第三，开展基于 GRPO 的优化训练与可靠性增强。在 SFT 达到可用基线后，将尝试引入 GRPO 等优化训练方法，重点提升模型在候选空间内推理的一致性与输出格式稳定性。训练目标将围绕“结构化可解析”“字段一致性”“根因与类型同时正确”等指标设定，并结合自动化评估脚本进行迭代，减少自由文本漂移、字段缺失与不一致输出等问题，提高模型在不同故障类型与不同证据分布下的鲁棒性。

第四，完善小模型召回与协同机制的可控性。后续将进一步优化小模型的统计特征设计与加权策略，系统比较不同 Top-K 候选规模对端到端效果与成本的影响，并引入更清晰的协同触发规则（如置信度阈值、候选差距、证据冲突等），以控制大模型调用频率。同时通过消融实验分析协同框架中各模块的贡献，明确性能瓶颈并指导改进方向。

第五，完成端到端系统工程实现与展示形态整理。在工程层面，将把数据抽取、样本构建、Top-K 召回、LLM 输入构建、模型训练与评估等环节进一步模块化与脚本化，统一配置管理与目录结构，使流程在本地与服务器环境能够一键运行并稳定复现。在此基础上整理演示接口与展示材料，支持从“输入故障事件”到“输出根因与故障类型”的完整演示，并输出可复用的实验记录与结果汇总，为论文撰写与答辩展示提供支撑。

第六，补齐评估体系与论文材料。后续将持续完善多指标评估与误差分析机制，除 Top-1/Top-K 命中外，进一步关注结构化可解析率、字段级准确率与完整匹配率等工程可用性指标，并结合典型成功/失败案例进行解释性分析。

五、　　进度安排：

时间安排　　工作内容

2025.12-2026.1 检索资料并充分翻译、阅读，深入了解本课题内容，初步形成研究方案。

2026.2-2026.3　对实验结果进行综合分析，得出结论。

2026.3-2026.4　开发软件系统。

2026.4-2026.5　撰写毕业论文。